



수학 및 순위 함수



06-4

수학 함수

- 여기에서는 ABS, SIGN, ROUND, RAND 등의 여러 수학 함수를 다룸
- 수학 함수는 대부분 입력값과 같은 데이터 유형으로 값을 반환하지만 LOG, EXP, SQRT 함수와 같은 함수는 입력값을 실수형인 float으로 자동 변환하고 반환함

▪ 절댓값을 구하는 함수 — ABS

- ABS 함수는 절댓값을 반환함
- 예를 들어 ABS 함수에 음수 -1.0을 입력하면 양수 1.0로 변환하는 반면 0이나 양수를 입력하면 변화가 없음
- 이때 ABS 함수의 인자에는 숫자가 아닌 수식을 입력해 절댓값을 반환받을 수도 있음



06-4

수학 함수

▪ 절댓값을 구하는 함수 — ABS

1. 다음 쿼리를 입력해 값을 확인해 보자.
다음은 음수, 0, 양수를 차례로 함수에 입력한 쿼리임.

Do it! ABS 함수에 입력한 숫자를 절댓값으로 반환

```
SELECT ABS(-1.0), ABS(0.0), ABS(1.0);
```

실행 결과

	ABS(-1.0)	ABS(0.0)	ABS(1.0)
▶	1.0	0.0	1.0

앞서 예로 들었던 내용 그대로 반환된 결과를 확인할 수 있음



06-4

수학 함수

■ 절댓값을 구하는 함수 — ABS

2. 이번에는 수식을 입력해 수식을 계산한 결과를 절댓값으로 반환하는 쿼리임.

Do it! ABS 함수에 입력한 수식의 결과를 절댓값으로 반환

```
SELECT a.amount - b.amount AS amount, ABS(a.amount - b.amount) AS  
abs_amount  
FROM payment AS a  
INNER JOIN payment AS b ON a.payment_id = b.payment_id - 1;
```

실행 결과

	amount	abs_amount
▶	2.00	2.00
	-5.00	5.00
	5.00	5.00
	-9.00	9.00
	5.00	5.00
	0.00	0.00

payment 테이블에서 payment_id가 현재보다 -1 값인
payment_id를 찾아서 amount값을 뺀 결과임.

결과에 따라 음수 또는 양수일 수 있음.

하지만 ABS 함수로 모두 양수로 결과를 반환함



06-4

수학 함수

▪ 절댓값을 구하는 함수 — ABS

3. ABS 함수는 자료형의 범위를 넘으면 오버플로가 발생함.
하지만 MySQL 서버에서는 BIGINT로 암시적 형 변환이 되어 값이 반환됨.

다음은 -2,147,483,648을 절댓값으로 변환하는 쿼리임.
인자로 전달한 값이 정수이므로 범위 외의 값임.

이러한 이유로 오류가 발생해야 하지만
자동으로 BIGINT로 형 변환된 것을 확인할 수 있음

Do it! 암시적 형 변환으로 오버플로 없이 절댓값을 반환

```
SELECT ABS(-2147483648);
```

실행 결과

	ABS(-2147483648)
▶	2147483648



06-4

수학 함수

▪ 절댓값을 구하는 함수 — ABS

- 결과 오른쪽에 [Field Types]를 클릭해 보자.
그러면 데이터 유형을 확인할 수 있는데
여기서는 자동으로 BIGINT로 형 변환된 것을 확인할 수 있음

#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	ABS(-2147483648)			BIGINT	binary	11	10	0



06-4

수학 함수

■ 양수 또는 음수 여부를 판단하는 함수 — SIGN

- SIGN 함수는 지정한 값이나 수식 결과값이 양수, 음수, 0인지를 판단하여 각각 1, -1, 0을 반환함

1. 다음과 같이 SIGN 함수에 숫자로 된 인수를 넣어 쿼리를 입력해 보자.

Do it! SIGN 함수로 입력한 숫자가 양수, 음수, 0인지를 판단

```
SELECT SIGN(-256), SIGN(0), SIGN(256);
```

실행 결과

	SIGN(-256)	SIGN(0)	SIGN(256)
▶	-1	0	1

결과를 살펴보면 -256은 음수를 의미하는 -1로,
256은 양수를 의미하는 1로 반환한 것을 확인할 수 있음



06-4

수학 함수

■ 양수 또는 음수 여부를 판단하는 함수 — SIGN

2. 이번에는 인자로 수식을 입력해
수식을 계산한 결과가 양수인지, 음수인지를 판단해 보자.

Do it! SIGN 함수로 수식의 결과가 양수, 음수, 0인지를 판단

```
SELECT a.amount - b.amount AS amount, SIGN(a.amount - b.amount) AS  
abs_amount  
FROM payment AS a  
INNER JOIN payment AS b ON a.payment_id = b.payment_id-1;
```

실행 결과		
	amount	abs_amount
▶	2.00	1
	-5.00	-1
	5.00	1
	-9.00	-1
	5.00	1

결과를 보면 amount 열의 계산 결과 값에 따라
음수이면 -1, 양수이면 1이 반환되는 것을 확인할 수 있음



06-4

수학 함수

▪ 천장값과 바닥값을 구하는 함수 — CEILING, FLOOR

- CEILING 함수는 천장값을 구함
- 천장값은 입력한 숫자보다 크거나 같은 최소 정수를 말함
예) 2.4는 3을 반환함
- 반대로 FLOOR 함수는 바닥값을 구하는데,
바닥값은 지정한 숫자보다 작거나 같은 최대 정수를 말함
예) 2.4는 2를 반환함



06-4

수학 함수

■ 천장값과 바닥값을 구하는 함수 — CEILING, FLOOR

1. 다음 쿼리를 통해 CEILING 함수를 이해해 보자.

Do it! CEILING 함수로 천장값 반환

```
SELECT CEILING(2.4), CEILING(-2.4), CEILING(0.0);
```

실행 결과

	CEILING(2.4)	CEILING(-2.4)	CEILING(0.0)
▶	3	-2	0

쿼리를 실행한 결과 2.4는 천장값으로 3을,
-2.4는 -2를, 0.0은 0을 출력하는 것을 확인할 수 있음



06-4

수학 함수

■ 천장값과 바닥값을 구하는 함수 — CEILING, FLOOR

2. 이번에는 FLOOR 함수를 이해해 보자.

Do it! FLOOR 함수로 바닥값 반환

```
SELECT FLOOR(2.4), FLOOR(-2.4), FLOOR(0.0);
```

쿼리를 실행한 결과 2.4는 바닥값으로 2를,
-2.4는 -3을 출력하는 것을 확인할 수 있음

실행 결과

	FLOOR(2.4)	FLOOR(-2.4)	FLOOR(0.0)
▶	2	-3	0



06-4

수학 함수

■ 반올림을 반환하는 함수 — ROUND

- ROUND 함수는 입력된 숫자의 반올림을 반환함
- ROUND 함수는 인자로 2개가 필요하며 기본 형식은 다음과 같음

ROUND 함수의 기본 형식

ROUND(숫자, 자릿수)

- 첫 번째 인수에는 반올림할 대상인 숫자값 또는 열 이름을 넣어 사용함
- 두 번째 인수에는 반올림하여 표현할 자릿수를 입력함



06-4

수학 함수

▪ 반올림을 반환하는 함수 — ROUND

- 예를 들어 첫 번째 인수는 123.1234이고, 두 번째 인수가 3이라면 ROUND 함수는 123.123을 반환함
- 자릿수의 데이터 유형은 정수임
- 또한 양수나 음수를 지정할 수 있는데 양수로는 소수부터 반올림하고, 음수로는 정수부터 반올림함



06-4

수학 함수

반올림을 반환하는 함수 — ROUND

1. 이제 쿼리를 입력해 ROUND 함수를 이해해 보자.

다음은 자릿수에 3을 입력해 소수 셋째 자리까지 표현함.
즉, 넷째 자리에서 반올림한 결과를 보여주는 쿼리임

Do it! ROUND 함수로 소수점 셋째 자리까지 반올림

```
SELECT ROUND(99.9994, 3), ROUND(99.9995, 3);
```

실행 결과

	ROUND(99.9994, 3)	ROUND(99.9995, 3)
▶	99.999	100.000



06-4

수학 함수

반올림을 반환하는 함수 — ROUND

2. 이번에는 자릿수에 양수와 음수를 입력해 보자.
결과에서 알 수 있듯 음수는 정수부터 반올림을 시작함.

Do it! ROUND 함수로 소수와 정수를 따로 반올림

```
SELECT ROUND(234.4545, 2), ROUND(234.4545, -2);
```

실행 결과

	ROUND(234.4545, 2)	ROUND(234.4545, -2)
▶	234.45	200

첫 번째는 소수점 둘째 자리로 반올림하여 234.45를,
두 번째는 10의 자리에서 반올림해 200을 반환함



06-4

수학 함수

반올림을 반환하는 함수 — ROUND

3. ROUND 함수는 자릿수에 음수를 전달할 때 입력된 정수의 자릿수보다 더 큰 자릿수(절대값으로 비교)를 입력하면 0을 반환함.

Do it! 정수 부분의 길이보다 큰 자릿수를 입력한 경우

```
SELECT ROUND(748.58, -1);  
SELECT ROUND(748.58, -2);  
SELECT ROUND(748.58, -4);
```

실행 결과

	ROUND(748.58, -1)
▶	750

	ROUND(748.58, -2)
▶	700

	ROUND(748.58, -4)
▶	0



06-4

수학 함수

▪ 반올림을 반환하는 함수 — ROUND

3. 첫 번째 쿼리는 1의 자리에서 반올림하여 750을,
두 번째 쿼리는 10의 자리에서 반올림하여 700을,
마지막 쿼리의 경우 1000의 자리에서 반올림을 해야 하는데

정수부인 748의 길이인 3보다 -4의 절댓값, 즉 4가 크므로 0을 반환함



06-4

수학 함수

로그를 구하는 함수 — LOG

- LOG 함수는 지정된 밑의 로그값을 구할 수 있음
- 로그 함수는 지수 함수의 역함수이므로 다음과 같은 관계를 나타냄

$$\log_b(a)=c \iff b^c=a$$

- LOG 함수의 기본 형식

LOG 함수의 기본 형식

LOG(로그 계산할 숫자 또는 식, 밑)



06-4

수학 함수

▪ 로그를 구하는 함수 — LOG

- LOG 함수의 첫 번째 인수에는 로그 계산할 숫자나 수식이 들어가는데, 결과는 실수형(float)으로 반환함
- 두 번째 인수로 밑을 입력할 수 있는데 말 그대로 로그의 밑을 설정하는 값임
- 밑은 생략이 가능함.
밑을 생략한 경우에는 자연 로그가 되며
이때 기본값으로 e가 설정됨



06-4

수학 함수

로그를 구하는 함수 — LOG

1. 다음과 같이 쿼리를 입력해 보자.
로그 10을 계산하는 쿼리임.

Do it! LOG 함수로 로그 10을 계산

```
SELECT LOG(10);
```

실행 결과

	LOG(10)
▶	2.302585092994046

이와 같이 로그의 밑을 생략해도 로그를 계산할 수 있는데,
이때 밑을 입력하지 않아도 자동으로 e가 기본값으로 설정된 것임



06-4

수학 함수

로그를 구하는 함수 — LOG

2. 이번에는 로그 10을 계산하는데 밑을 5로 설정해 계산해 보자

Do it! LOG 함수로 로그 10의 5를 계산

```
SELECT LOG(10, 5);
```

실행 결과

	LOG(10, 5)
▶	0.6989700043360187



06-4

수학 함수

▪ e의 n 제곱값을 구하는 함수 — EXP

- EXP 함수는 e의 n 제곱값을 반환함
- 여기서 e는 앞서 LOG 함수에서 만난 로그 함수의 기본 밑과 동일함
- 이 함수 역시 결과는 실수형(float)로 반환함
- 이때 실수 다음은 EXP 함수의 기본 형식임

EXP 함수 기본 형태

EXP(지수 계산할 숫자 또는 식)



06-4

수학 함수

▪ e의 n 제곱값을 구하는 함수 — EXP

1. 다음 쿼리로 EXP(1.0)를 계산해 보자.

Do it! EXP 함수로 지수 1.0을 계산

```
SELECT EXP(1.0);
```

실행 결과

	EXP(1.0)
▶	2.718281828459045

EXP(1.0)은 $e^{1.0}$ 이므로 결과값으로 2.718281828459045을 얻음



06-4

수학 함수

▪ e의 n 제곱값을 구하는 함수 — EXP

2. 이번에는 다음 쿼리로 EXP(10)을 계산해 보자

Do it! EXP 함수로 지수 10을 계산

```
SELECT EXP(10);
```

실행 결과

	EXP(10)
▶	22026.465794806718



06-4

수학 함수

▪ e의 n 제곱값을 구하는 함수 — EXP

2. 아마 계산한 결과값이 제대로 된 것인지 의심스러울 수 있음.
그럴 때는 LOG 함수가 EXP 함수의 역함수임을 기억하자.

다음은 자연 로그 20을 계산한 다음,
그 값을 EXP 함수로 지수 계산을 하고 거꾸로도 계산해 본 쿼리임

Do it! LOG 함수와 EXP 함수로 결과 확인

```
SELECT EXP(LOG(20)), LOG(EXP(20));
```

실행 결과

	EXP(LOG(20))	LOG(EXP(20))
▶	19.999999999999996	20



06-4

수학 함수

▪ e의 n 제곱값을 구하는 함수 — EXP

2. 우리가 아는 이론으로는
두 함수 곱셈값 모두 20을 반환해야 하지만 차이가 있음.

그 이유는 실수 변환에서 반올림의 처리 때문임.
그러므로 사실상 결과는 같다고 볼 수 있음



06-4

수학 함수

▪ 거듭제곱값을 구하는 함수 — POWER

- POWER 함수는 거듭제곱값을 구함
- 기본 형식을 보면 이 함수는 밑인 숫자 또는 수식과 지수를 인자로 입력받음

POWER 함수의 기본 형식

POWER(숫자 또는 수식, 지수)

- POWER 함수의 첫 번째 인자는 정수 또는 실수형(float)만 입력받을 수 있음
- 두 번째 인자는 거듭제곱할 정수 또는 실수를 입력 받음



06-4

수학 함수

■ 거듭제곱값을 구하는 함수 — POWER

- 다음은 2의 3 거듭제곱, 2의 10 거듭제곱, 2.0의 3 거듭제곱을 POWER 함수로 계산한 쿼리임

Do it! POWER 함수로 거듭제곱 계산

```
SELECT POWER(2,3), POWER(2,10), POWER(2.0, 3);
```

실행 결과

	POWER(2,3)	POWER(2,10)	POWER(2.0, 3)
▶	8	1024	8

- 결과를 보면 첫 번째 열에는 2^3 인 결과 8, 두 번째 열에는 2^{10} 거듭제곱인 1024, 세 번째 열에서는 2.0^3 으로 8이 반환됨



06-4

수학 함수

■ 제곱근을 구하는 함수 — SQRT

- SQRT 함수는 숫자나 수식을 인자로 받아 제곱근을 반환함
- 이때 실수형(float)을 입력받음

SQRT 함수의 기본 형식

SQRT(숫자 또는 수식)

- 다음 쿼리는 1의 제곱근, 10의 제곱근을 반환함

Do it! SQRT 함수로 제곱근 계산

```
SELECT SQRT(1.00), SQRT(10.00);
```

실행 결과

	SQRT(1.00)	SQRT(10.00)
▶	1	3.1622776601683795

- 제곱근을 계산할 때 양수 값을 넣어야 하며
음수 값 입력으로 인해 결과가 음수 값일 경우 NULL이 반환됨



06-4

수학 함수

▪ 난수를 구하는 함수 — RAND

- RAND 함수는 0부터 1 사이의 실수형 난수를 반환함

RAND 함수의 기본 형식

RAND(인자)

- 인자로 전달하는 값의 데이터 유형은 tinyint, smallint, int임
- 만약 인자를 지정하지 않으면 임의로 초깃값을 설정함



06-4

수학 함수

▪ 난수를 구하는 함수 — RAND

1. RAND 함수의 인자는 난수 종류를 결정하는 값이며 같은 인자를 설정하면 RAND 함수는 같은 결과를 반환함.

다음과 같이 결과 비교를 위해 RAND 함수를 3번 입력해 보자

Do it! RAND 함수로 난수 계산

```
SELECT RAND(100), RAND(), RAND();
```

실행 결과

	RAND(100)	RAND()	RAND()
▶	0.17353134804734155	0.0726371417498562	0.9080051518119919



06-4

수학 함수

▪ 난수를 구하는 함수 — RAND

1. 난수를 구하는 RAND 함수가 인자에 의해 같은 난수 종류를 반환한다는 것이 당황스럽지만,

쿼리를 실행한 컴퓨터마다 다른 결과를 반환하므로
어찌 보면 난수를 제대로 반환한다고 보아도 무방함.

그런데 난수라고 설명했는데도 계속 같은 값이 반환되는 이유는
현재 쿼리 창에서 처음 함수를 실행할 때
임의로 정해진 초깃값을 계속 재사용하기 때문임.

다른 쿼리 창을 열어 쿼리를 실행해 보면
다른 결과값이 검색되는 것을 확인할 수 있음



06-4

수학 함수

▪ 난수를 구하는 함수 — RAND

2. 다음은 인자를 설정하지 않은 채로
WHILE 문을 사용하여 난수를 4회 생성한 쿼리임.

이 경우 RAND 함수 인자에 아무것도 전달하지 않았으므로
데이터베이스가 설정한 임의의 값으로 난수 종류를 보여 주므로
실행할 때마다 다른 난수를 볼 수 있음.

MySQL에서 WHILE을 사용하려면
스토어드 프로시저를 생성해서 사용해야 함.

스토어드 프로시저를 아직 배우지 않았지만
우선 해당 쿼리를 따라 작성해 실습을 진행함



06-4

수학 함수

▪ 난수를 구하는 함수 — RAND

2.

Do it! 인수가 없는 RAND 함수로 난수 계산

```
DELIMITER $$  
CREATE PROCEDURE rnd()  
BEGIN  
    DECLARE counter INT;  
    SET counter = 1;  
  
    WHILE counter < 5 DO  
        SELECT RAND() Random_Number;  
        SET counter = counter + 1;  
    END WHILE;  
END $$  
DELIMITER ;  
  
call rnd();
```

실행 결과

	Random_Number
▶	0.9633106123314338



06-4

수학 함수

■ 삼각 함수 — COS, SIN, TAN, ATAN

- 삼각 함수는 COS 함수부터 DEGREES 함수에 이르기까지 다양하지만 COS, SIN, TAN, ATAN 함수를 간단히 살펴보고자 함
- 모든 삼각 함수는 실수형 인자를 받음
- 기본 형식은 COS 함수만 살펴봄

COS 함수의 기본 형식

COS(숫자 또는 식)



06-4

수학 함수

■ 삼각 함수 — COS, SIN, TAN, ATAN

1. 바로 쿼리를 입력하여 COS 함수의 결과를 살펴보자.
다음은 COS에 14.78(=14.78°)을 입력하여 얻은 결과임

Do it! COS 함수 계산

```
SELECT COS(14.78);
```

실행 결과

	COS(14.78)
▶	-0.5994654261946543

1. SIN 함수, TAN 함수, ATAN 함수에도
다음과 같이 인자를 넣어 차례로 실행해 보자

Do it! SIN 함수 계산

```
SELECT SIN(45.175643);
```

실행 결과

	SIN(45.175643)
▶	0.929607286611012



06-4

수학 함수

삼각 함수 — COS, SIN, TAN, ATAN

2.

Do it! TAN 함수 계산

```
SELECT TAN(PI()/2), TAN(.45)
```

실행 결과

	TAN(PI()/2)	TAN(.45)
▶	1.633123935319537e16	0.4830550656165784



06-4

수학 함수

삼각 함수 — COS, SIN, TAN, ATAN

2.

Do it! ATAN 함수 계산

```
SELECT ATAN(45.87) AS atanCalc1,  
       ATAN(-181.01) AS atanCalc2,  
       ATAN(0) AS atanCalc3,  
       ATAN(0.1472738) AS atanCalc4,  
       ATAN(197.1099392) AS atanCalc5;
```

실행 결과

	atanCalc1	atanCalc2	atanCalc3	atanCalc4	atanCalc5
▶	1.548999038348081	-1.5652718263440326	0	0.1462226769376524	1.5657230594360985



06-5

순위 함수

- **삼각 함수 — COS, SIN, TAN, ATAN**
 - 순위 함수는 조회 결과에 순위를 부여함
 - MySQL은 순위 함수로 ROW_NUMBER, RANK, DENSE_RANK, NTILE 등을 제공함
 - 순위 함수는 전체 데이터에 순위를 부여할 수도 있고, PARTITION 옵션을 함께 사용해 사용자가 설정한 그룹에 따라 그룹 내에서만 순위를 부여할 수도 있음



06-5

순위 함수

- **유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER**
 - ROW_NUMBER 함수는 모든 행에 유일한 값으로 순위를 부여함
 - 다시 말해 함수를 실행한 결과에는 같은 순위가 없음
 - 만약 같은 순위라면 정렬 순서에 따라 순위를 다르게 부여하여 반환함



06-5

순위 함수

▪ 유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER

▪ ROW_NUMBER 함수의 기본 형식

ROW_NUMBER 함수의 기본 형식

```
ROW_NUMBER() OVER([PARTITION BY 열] ORDER BY 열)
```

- ROW_NUMBER 함수는 순위를 부여하기 위한 정렬 조건으로 OVER(ORDER BY 열)을 명시하여 오름차순 또는 내림차순으로 해당 열 데이터의 순위를 부여함
- 이때 데이터 그룹별 순위를 부여하고 싶다면 **PARTITION BY** 열 옵션을 사용함



06-5

순위 함수

▪ 유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER

1. 다음은 payment 테이블에서 customer_id, amount 열을 조회하는데,

먼저 customer_id 열을 그룹화한 뒤,
그룹별로 amount의 값을 합한 결과를 내림차순 정렬한 결과에 따라
ROW_NUMBER 함수로 순위를 부여하는 쿼리임

Do it! ROW_NUMBER 함수로 순위 부여

```
SELECT ROW_NUMBER() OVER(ORDER BY amount DESC) AS num,  
customer_id, amount  
FROM (  
    SELECT customer_id, SUM(amount) AS amount  
    FROM payment GROUP BY customer_id  
) AS x;
```

실행 결과

	num	customer_id	amount
▶	1	526	221.55
	2	148	216.54
	3	144	195.58
	4	137	194.61
	5	178	194.61
	6	459	186.62
	7	469	177.60
	8	468	175.61
	9	236	175.58
	10	181	174.66



06-5

순위 함수

▪ 유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER

1. 결과를 살펴보면 amount값이 클수록 순위가 높음.
동일한 amount값에도 순위가 다른 점도 확인하자.

순위를 저장한 num열의 4, 5위를 보면
amount값이 같음에도 순위가 다른 것을 확인할 수 있음.

같은 순위 데이터의 경우 어떤 데이터에 어떤 순위를 결정할지는
데이터 정렬 순서에 따라 달라짐.

여기서는 customer_id 숫자가 작을수록 우선순위를 부여받음



06-5

순위 함수

▪ 유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER

2. 만약 동점에 대해서 순위를 MySQL이 임의로 부여하지 않게 하려면 ORDER BY 문에 정렬 조건을 추가함.

다음 쿼리는 앞선 실습과 쿼리가 거의 동일하나
ORDER BY 문에 customer_id 열을 내림차순으로 정렬하는 조건을 추가함.

이 쿼리는 amount 열의 데이터가 같으면
customer_id 열의 데이터가 더 클수록 우선순위를 부여받음



06-5

순위 함수

■ 유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER

2.

Do it! 내림차순 정렬한 결과에 ROW_NUMBER 함수로 순위 부여

```
SELECT ROW_NUMBER() OVER(ORDER BY amount DESC,  
customer_id DESC)  
AS num, customer_id, amount  
FROM (  
    SELECT customer_id, SUM(amount) AS amount  
    FROM payment GROUP BY customer_id  
) AS x;
```

실행 결과

num	customer_id	amount
1	526	221.55
2	148	216.54
3	144	195.58
4	178	194.61
5	137	194.61
6	459	186.62
7	469	177.60
8	468	175.61
9	236	175.58
10	181	174.66

앞선 실습과는 반대의 결과가 출력됨



06-5

순위 함수

▪ 유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER

3. 전체 데이터가 아니라 그룹별로 순위를 부여해야 할 때가 있음.

예를 들어 전교생 대상으로 석차를 구할 때와
학년별 석차를 구할 때를 생각하면 됨.

그룹별 순위를 부여하려면 PARTITION 문을 사용해야 함.

다음은 앞에서 실습한 쿼리를 다시 변경하여
staff_id 열을 그룹화한 결과에 순위를 부여하는 쿼리임.
staff_id 열을 그룹화하기 위해 다음과 같이 PARTITION BY staff_id를 추가함



06-5

순위 함수

■ 유일한 값으로 순위를 부여하는 함수 — ROW_NUMBER

3.

Do it! PARTITION BY 문으로 사용해 그룹별 순위 부여

```
SELECT staff_id,  
ROW_NUMBER() OVER(PARTITION BY staff_id ORDER BY amount DESC,  
customer_id ASC)  
AS num, customer_id, amount  
FROM (  
    SELECT customer_id, staff_id, SUM(amount) AS amount  
    FROM payment GROUP BY customer_id, staff_id  
) AS x;
```

실행 결과

	staff_id	num	customer_id	amount
▶	1	1	176	126.74
	1	2	137	115.77
	1	3	526	111.79
	1	4	178	108.81
	1	5	459	108.78
	1	596	281	15.95
	1	597	248	15.93
	1	598	344	12.95
	1	599	320	10.95
	2	1	187	110.81
	2	2	148	110.78
	2	3	526	109.76
	2	4	522	108.79
	2	5	211	108.77
	2	6	309	104.81

결과를 보면 staff_id 열이 1과 2 그룹으로 묶였는데,
각 그룹마다 순위가 부여되었음을 확인할 수 있음



06-5

순위 함수

▪ 우선순위를 고려하지 않고 순위를 부여하는 함수 — RANK

- RANK 함수는 ROW_NUMBER 함수와 비슷하지만 같은 순위를 처리하는 방법은 다름
- RANK 함수를 활용할 때에는 우선순위를 따지지 않고 같은 순위를 부여함

▪ RANK 함수의 기본 형식

RANK 함수의 기본 형식

```
RANK() OVER([PARTITION BY 열] ORDER BY 열)
```




06-5

순위 함수

- **우선순위를 고려하지 않고 순위를 부여하는 함수 — RANK**
 - RANK 함수도 순위를 부여하는 함수임
 - ROW_NUMBER 함수와 마찬가지로 순위를 부여하기 위한 정렬 조건으로 OVER(ORDER BY 열)을 명시하여 해당 열을 기준으로 오름차순 또는 내림차순으로 해당 열 데이터의 순위를 부여함
 - 이때 데이터 그룹별 순위를 부여하고 싶다면 PARTITION BY 열 옵션을 사용함



06-5

순위 함수

■ 우선순위를 고려하지 않고 순위를 부여하는 함수 — RANK

- 다음은 payment 테이블에서 amount 열을 내림차순 정렬하여 RANK 함수로 순위를 부여하는 쿼리임

Do it! RANK 함수로 순위 부여

```
SELECT RANK() OVER(ORDER BY amount DESC) AS num, customer_id,  
amount  
FROM (  
    SELECT customer_id, SUM(amount) AS amount  
    FROM payment GROUP BY customer_id  
) AS x;
```

실행 결과

	num	customer_id	amount
▶	1	526	221.55
	2	148	216.54
	3	144	195.58
	4	137	194.61
	4	178	194.61
	6	459	186.62
	7	469	177.60
	8	468	175.61
	9	236	175.58
	10	181	174.66



06-5

순위 함수

▪ 우선순위를 고려하지 않고 순위를 부여하는 함수 — RANK

- 결과를 살펴보면 amount값이 같은 경우 같은 순위를 부여한 것을 알 수 있음
- 순위를 저장한 num 열의 4위를 보면 이 내용을 확인할 수 있음
- 공동 순위가 있으면 그다음 순위는 같은 순위에는 데이터 개수만큼 건너뛴 순위가 부여됨
- 때문에 여기서는 4위 다음에 6위로 매겨짐
- 만약 1위가 3개면 그다음 순위는 2가 아니라 4가 됨
- RANK 함수 역시 그룹별로 순위를 부여할 때는 PARTITION 문을 사용하면 되는데, 사용 방법은 ROW_NUMBER 함수와 동일함



06-5

순위 함수

▪ 건너뛰지 않고 순위를 부여하는 함수 — DENSE_RANK

- DENSE_RANK 함수는 RANK 함수와 동일하지만 RANK 함수와 달리 같은 순위에 있는 데이터 개수를 무시하고 순위를 매긴다는 점은 다름
- 예를 들어 1위가 3개여도 그다음 데이터는 4위가 아닌 2위가 됨
- DENSE_RANK 함수의 기본 형식

DENSE_RANK 함수의 기본 형식

```
DENSE_RANK() OVER([PARTITION BY 열] ORDER BY 열)
```



06-5

순위 함수

■ 건너뛰지 않고 순위를 부여하는 함수 — DENSE_RANK

- 다음은 payment 테이블에서 amount 열을 내림차순 정렬하여 DENSE_RANK 함수로 순위를 부여하는 쿼리임

Do it! DENSE_RANK 함수로 순위 부여

```
SELECT DENSE_RANK() OVER(ORDER BY amount  
DESC)  
AS num, customer_id, amount  
FROM (  
    SELECT customer_id, SUM(amount) AS  
amount  
    FROM payment GROUP BY customer_id  
) AS x;
```

실행 결과

	num	customer_id	amount
▶	1	526	221.55
	2	148	216.54
	3	144	195.58
	4	137	194.61
	4	178	194.61
	5	459	186.62
	6	469	177.60
	7	468	175.61
	8	236	175.58
	9	181	174.66



06-5

순위 함수

- **건너뛰지 않고 순위를 부여하는 함수 — DENSE_RANK**
 - 결과를 살펴보면 amount값이 같은 경우 같은 순위를 부여한 것을 알 수 있음
 - 그리고 순위를 저장한 num 열의 4위를 보면 RANK 함수와 달리 그다음 순위는 건너뛰지 않고 바로 5위가 부여된 것을 확인할 수 있음



06-5

순위 함수

■ 그룹 순위를 부여하는 함수 — NTILE

- NTILE 함수는 인자로 지정한 개수만큼 데이터 행을 그룹화한 다음 각 그룹에 순위를 부여함
- 각 그룹은 1부터 순위가 매겨지며, 이때 순위는 각 행의 순위가 아니라 행이 속한 그룹의 순위임
- NTILE 함수 기본 형식
(이때, NTILE 함수의 인자는 반드시 정수로 입력해야 함)

NTILE 함수의 기본 형식

```
NTILE(정수) OVER([PARTITION BY 열] ORDER BY 열)
```



06-5

순위 함수

■ 그룹 순위를 부여하는 함수 — NTILE

- 다음은 payment 테이블에서 amount 열을 내림차순 정렬한 다음, 그룹에 대한 순위를 부여하는 쿼리임

Do it! 내림차순으로 정렬한 결과에 NTILE 함수로 순위 부여

```
SELECT NTILE(100) OVER(ORDER BY amount DESC)
AS num, customer_id, amount
FROM (
    SELECT customer_id, SUM(amount) AS
amount
    FROM payment GROUP BY customer_id
) AS x;
```

실행 결과

	num	customer_id	amount
▶	1	526	221.55
	1	148	216.54
	1	144	195.58
	1	137	194.61
	1	178	194.61
	1	459	186.62
	2	469	177.60
	2	468	175.61
	2	236	175.58
	2	181	174.66



06-5

순위 함수

■ 그룹 순위를 부여하는 함수 — NTILE

- 결과를 보면 전체 행을 100개의 그룹으로 쪼갠 뒤, 각 행마다 속한 그룹의 순위를 부여한 것을 확인할 수 있음
- NTILE 함수는 전체 행을 균등하게 나누어서 1순위 그룹, 2순위 그룹에 차등으로 혜택을 지급할 때 활용하기 좋은 함수임